
Contents

Istruzioni del RISC-V ISA	2
Istruzioni aritmetiche	2
Addizione	2
Addizione tra registro e intero	2
Sottrazione	2
Cambio di segno	2
Copia di un valore da un registro all'altro	2
Moltiplicazione	3
Divisioni	3
Caricamento di un numero nel registro	3
Istruzioni di accesso alla memoria	3
Istruzione di caricamento dalla memoria(RV32I)	3
Istruzioni di scrittura sulla memoria(RV32I)	5
Allineamento degli indirizzi	5
Istruzioni di caricamento della memoria senza indirizzi	6
Operandi immediati e costanti	7
Somma immediata	7
Operazioni logiche(shift)	7
Shift logico a sinistra	7
Moltiplicazioni per 2	8
Shift logico a destra	8
Divisioni per 2	8
Shift aritmetico a destra	8
Istruzioni per l'algebra di Boole	9
Salti condizionati	9
Salto incondizionato	10
Costrutto if-then-else	10
Costrutto for	11
SLT	11
Pseudo istruzioni	12
Copia di registro in un altro	12

Istruzioni del RISC-V ISA

Si distinguono in user e superuser. Le istruzioni che vedremo sono tutte a livello user. Sono sempre lunghe 32 bit anche ad estensioni a 64 bit. Alcune sono pseudo-istruzioni che vengono poi convertite dall'assemblatore.

Istruzioni aritmetiche

Le istruzioni aritmetiche si aspettano esattamente 3 operandi che sono 3 registri in quanto l'ALU supporta solo operazioni sui registri. I numeri sono rappresentati in complemento a due

Addizione

$a = b + c$ in RISC-V diventa `add a, b, c`

Addizione tra registro e intero

Questo operatore somma il valore di un registro ad un numero costante.

Può anche essere usato per impostare un registro ad un determinato valore, ad esempio:

`addi x5, x0, 41` # Carica 41 nel registri x5

Sottrazione

$a = b - c$ in RISC-V diventa `sub a, b, c`

Cambio di segno

Usando il registro x0 e l'istruzione `sub` possiamo creare il negato di un numero(per ipotesi contenuto in x19): `sub x19, x0, x19` cioè $a = 0 - a$

Copia di un valore da un registro all'altro

Usando il registro x0 possiamo copiare il valore da un registro all'altro: `add x19, x0, x18` cioè $a = 0 + b$. In altri ISA c'è `move` per fare questo senza x0 ma non sono architetture RISC(numero di istruzioni minimo)

Moltiplicazione

Per eseguire moltiplicazioni in RISC-V, possiamo usare l'operatore `mul`.

`mul t0, x4, x5` cioè moltiplica `x4` e `x5` e salva il risultato in `t0`

Esiste anche la versione unsigned `mulh`

Divisioni

Per eseguire divisioni in RISC-V, possiamo usare l'operatore `div`.

`div t0, x4, x5` cioè dividi `x4` e `x5` e salva il risultato in `t0`.

Possiamo anche ottenere il resto con l'operatore `rem`

La sintassi è la stessa di `div`. Esistono anche le corrispettive versioni unsigned di questi comandi.

Caricamento di un numero nel registro

Si usa l'operatore `li` load immediate:

`li x5, 60` # Carica 60 in `x5`

Tutte le operazioni immediate sono pseudo istruzioni RARS che vengono convertite in istruzioni basiche del RISC-V Ad esempio questa viene convertita come `addi` del valore immediate nel registro.

Istruzioni di accesso alla memoria

La memoria centrale si indirizza al byte. Può essere astratta come un vettore monodimensione. Si accede alle posizioni attraverso l'indirizzo(byte). Questo indirizzo non è memorizzato in memoria ma è il metodo d'indirizzamento della memoria. I valori vengono memorizzati nella cella all'indirizzo `n` (gli indirizzi non sono memorizzati nella cella).

La memoria centrale serve per le variabili che non servono immediatamente al codice assembly (dato che i registri sono pochi) oppure per contenere array

Istruzione di caricamento dalla memoria (RV32I)

Per caricare valori dalla memoria ai registri, usiamo l'istruzione `lw` cioè load word. Questa istruzione copia una word (4 byte) in un registro. La sintassi è la seguente: `lw registro_destinazione, offset(indirizzo base)`. Ad esempio: `lw x5, 12(x21)`.

L'indirizzo della memoria centrale è composto da:

- Indirizzo di base(contenuto in un registro per l'uso con l'operatore `lw`)
- L'offset(o scostamento) cioè quanto sommare all'indirizzo base. Può anche essere negativo(da -2048 a +2047)

Questo meccanismo dell'offset è molto utile per accedere agli array. Infatti un array ha un indirizzo base e un offset. Ad esempio: `a[3]` vuol dire l'indirizzo base di `a` e poi l'offset per arrivare alla posizione $3(4 * 3)$ in quanto la dimensione di un `int` è 4 byte

Se volessimo leggere un singolo `int`(no array), useremmo `offset=0`

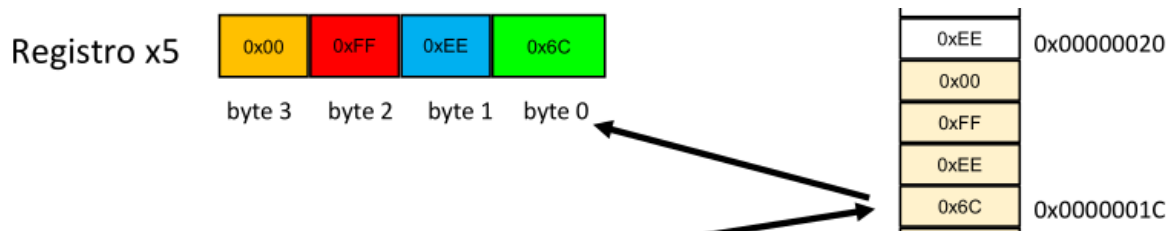
L'indirizzo che verrà usato sarà quindi indirizzo base + offset.(calcolo fatto dall'ALU) A partire da questo indirizzo verranno letti 4 byte(una word) contigui(infatti gli array sono porzioni di memoria contigua) e messi poi in un singolo registro(32 bit, 4 byte)

Questa operazione copia i valori, non li cancella.

Ordine dei byte

byte 3	byte 2	byte 1	byte 0
--------	--------	--------	--------

Il bit più significativo è quindi a sinistra. Cioè l'ordine è **little endian**. Ciò comporta che al bit meno significativo(byte 0) viene caricato il valore all'indirizzo(indirizzo base + offset) poi negli altri bit(1,2,3) verranno messi i 3 byte successivi. Cioè la prima cella letta è la meno significati. L'opposto di questo ordine è **big endian**



Altre istruzioni(la sintassi è analoga a `lw`):

- `lb` per caricare un singolo byte nel byte meno significativo del registro. Viene fatta l'estensione del segno
- `lh` per caricare 2 byte nel registro. Viene fatta l'estensione del segno
- `lbu` per caricare un singolo byte nel byte meno significativo del registro senza estensione del segno

-
- `lhu` per caricare un half word nel registro senza estensione del segno

L'estensione del segno permette di rappresentare numeri da 8 o 16 bit su 32 bit. Questo si fa riempiendo di 0 i numeri mancanti nel caso di numeri positivi o riempiendo di 1 nel caso di numeri negativi(cioè riempie tutti i bit rimanti con il bit più significativo).

Con le ultime due istruzioni possiamo creare numeri senza segno(unsigned) cioè il riempimento viene fatto con 0

Istruzione di caricamento dalla memoria(RV64I) Nelle estensioni a 64 bit(non in quelle a 32) esiste anche l'istruzione `ld` load doubleword per caricare 64 bit in un registro. Il funzionamento è analogo a `lw`. Non lo useremo

Istruzioni di scrittura sulla memoria(RV32I)

L'istruzione `sw` store word copia un dato da un registro sulla memoria. La sintassi è `sw registro_che_contiene_il_valore_da_scrivere, offset(registro_che_contiene_indirizzo)`. La sintassi dell'indirizzo è uguale a quello di `lw`

Questa istruzione prende un valore da un registro e lo scrive nell'indirizzo `indirizzo_base+offset`.

Il valore nel registro non viene mai cancellato.

L'ordine rimane little endian. Il byte 0 del registro va nell'indirizzo specificato da `indirizzo_base+offset`

Altre istruzioni:

- `sh` per caricare una half-word partendo dal meno significativo
- `sb` per caricare il byte meno significativo di un registro nella memoria centrale

Istruzioni di scrittura sulla memoria(RV64I) Nelle estensioni a 64 bit(non in quelle a 32) esiste anche l'istruzione `sd` load doubleword per caricare 64 bit in un registro. Il funzionamento è analogo a `sw`. Non lo useremo

Queste istruzioni sono utili per memorizzare strutture dati complesse come array o strutture che non possono stare nei registri.

Allineamento degli indirizzi

La memoria è indirizzata al byte, per questa ragione gli indirizzi dovrebbero essere allineati in questo modo:

Istruzione	Allineamento
<code>lw e sw</code>	4
<code>lh, lhu e sh</code>	2

RISC-V non ha vincoli di allineamento però è molto meno efficiente se non allineiamo gli indirizzi (l'emulatore però lo vieta).

Allineato vuol dire che l'indirizzo è multiplo del numero in colonna a destra.

Istruzioni di caricamento della memoria senza indirizzi

Usiamo l'istruzione `la` load address che carica in un registro l'indirizzo di una word salvata nei data.

Esempio:

```
.globl _start
.data
    v1: .word 8 2 3 # Offset di 8
    v2: .word 9 # Offset di 9
    v3: .word 0 # Offset di 0
    v4: .byte 1 2
.text
_start:
    la t0, v1
    lw t1, 1(t0)    # Non è possibile in posizione 1 perchè non è allineato alla
    lw t1, 0(t0)    # Carica 8 nel registro t1
    lw t2, 4(t0)    # Carica 9(v1 + 4 = v2) nel registro t2
    lw t3, 8(t0)

    la t0, v2
    lw t4, (v2)

    la t0, v4
    lb t5, (t0)
    lb t6, 1(t0) # Dato che sto caricando un byte sono allineato se metto 1
```

Successivamente possiamo usare l'indirizzo per le normali istruzioni load word. Gli indirizzi nei data sono contigui.

L'istruzione `la` è una pseudo-istruzione che viene convertita in `auipc` e poi in `add` perché il valore dell'indirizzo è molto grande per stare nel formato delle istruzioni RISC-V quindi va spezzata in più istruzioni.

Operandi immediati e costanti

Somma immediata

`addi` è l'istruzione di somma immediata dove un valore è una costante. La sintassi è `addi REGISTRO_DA_SALVARE, REGISTRO_DA_SOMMARE, COSTANTE` Esempio `addi x5, x5, 10` cioè salviamo in `x5` la somma tra `x5` e 10.

La costante ha un range (-2048, 2047)

Notiamo che le somme possono avere solo due addendi perché l'ALU ha solo due ingressi.

Con questa operazione svolgiamo anche le sottrazioni.

Tutte le operazioni immediate sono pseudo istruzioni RARS che vengono convertite in istruzioni basiche del RISC-V

Operazioni logiche(shift)

Shift logico a sinistra

Dato un valore `0011` se traslo verso sinistra, sposto il bit più significativo verso sinistra cioè lo elimino e tutti i bit si spostano verso sinistra $x_i = x_{i-1}$. Il bit meno significativo diventa 0. `0011` shiftato di una posizione diventa `0110`, di due `1100`, ecc...



In RISC-V si può realizzare come: `sll REGISTRO_DESTINAZIONE, REGISTRO_DA_SHIFTARE, REGISTRO_SHIFT`

Dove il registro `shift` indica di quante posizioni fare lo shift. Di questo registro vengono considerati solo i 5 bit meno significativi (formato `I`).

Il registro da shiftare non viene modificato. Solo il registro destinazione viene modificato.

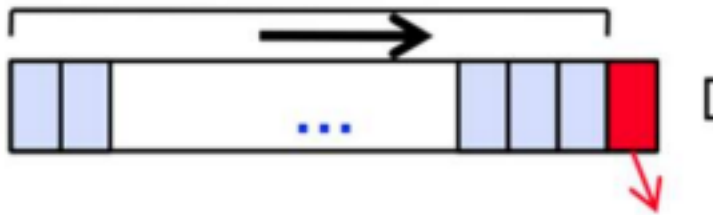
Esiste anche la versione immediate per evitare l'uso di `REGISTRO_SHIFT`.

Moltiplicazioni per 2

`slli x7, x7, 1`. Moltiplica il numero contenuto in x7 per 2^1 . Ad esempio `slli x7, x7, 2` moltiplica x7 per 2^2 e così via.

Shift logico a destra

La stessa operazione può essere fatta verso destra usando l'operatore `srl`. Il bit meno significativo perde valore e gli altri traslano verso destra. La sintassi e funzionamento è come prima.

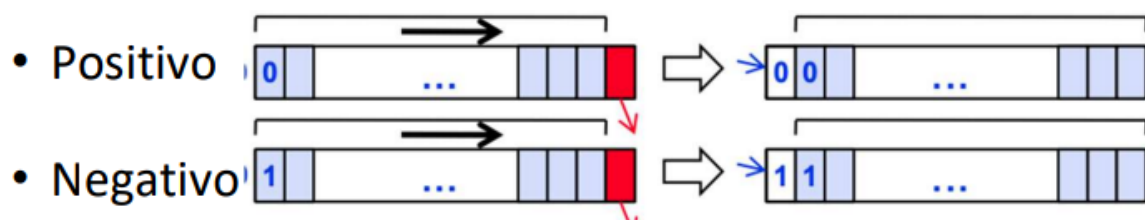


Divisioni per 2

`srl x7, x7, 1`. Divide il numero contenuto in x7 per 2^1 . Ad esempio `srl x7, x7, 2` divide x7 per 2^2 e così via.

Shift aritmetico a destra

Viene usato per evitare di perdere il segno di un numero in complemento a due. Fa uno shift a destra ma replica il bit più significativo per evitare la perdita di segno.



è disponibile anche la versione immediata. Ovviamente verso sinistra non esiste perché non abbiamo il segno.

Istruzioni per l'algebra di Boole

AND Sintassi: `and REGISTRO_RISULTATO, R1, R2` oppure immediata `andi REGISTRO_RISULTATO, R1, NUMERO`

Facciamo l'AND logico tra due registri bit a bit, il risultato viene salvato nel primo operando (REGISTRO_RISULTATO). Nella versione immediata viene fatta questa operazione con NUMERO.

OR Sintassi: `or REGISTRO_RISULTATO, R1, R2` oppure immediata `ori REGISTRO_RISULTATO, R1, NUMERO`

Il funzionamento è medesimo all'AND

XOR Sintassi: `xor REGISTRO_RISULTATO, R1, R2` oppure immediata `xori REGISTRO_RISULTATO, R1, NUMERO`

Il funzionamento è medesimo all'AND

Queste istruzioni servono per svolgere operazioni con maschere di bit. Ad esempio con l'AND possiamo solo selezionare alcuni bit ($1 \text{ AND } 0 = 0$, possiamo azzerare opportune posizioni). Oppure con l'OR possiamo aggiungere valori 1 ad alcune posizioni arbitrarie (infatti $1 \text{ OR } 0 = 1$, possiamo aggiungere 1 ad opportune posizioni).

NOT Sintassi: `not R1, R2`

Viene fatto il NOT di R1 bit a bit e il risultato salvato in R2.

Salti condizionati

Queste istruzioni permettono di saltare ad un'altra istruzione variando il program counter (utilizzando `addi`), se una condizione è vera o falsa. Ad esempio se l'else si trova due posizioni dopo al salto, viene fatto `addi PC, zero, 0x08` cioè aumenta il program counter di 8 byte (2 istruzioni).

Queste istruzioni servono anche per fare cicli `for` e `while`.

- `beq rs1, rs2, L1` dove i primi due argomenti sono registri di cui fare la comparazione e L1 è l'etichetta dove il programma deve saltare. `beq` significa "Branch if Equal" e fa il salto all'etichetta L1 se e solo se $rs1 == rs2$ (confronto bit per bit)

-
- bne, significa “Branch If Not Equal”, salta a L1 se e solo se $rs1 \neq rs2$ (confronto bit per bit)
 - blt “Branch if Less Than” salta a L1 se e solo se $rs1 < rs2$
 - bge, “Branch if Greater than or Equal”, salta a L1 se e solo se $rs1 \geq rs2$

Esistono anche le versioni unsigned: - bltu, salta a L1 se e solo se $rs1 < rs2$ senza contorziare il segno - bgeu, salta a L1 se e solo se $rs1 \geq rs2$ senza controllare il segno

La sintassi per tutte queste istruzioni di salto è uguale tra loro.

Salto incondizionato

Per saltare in maniera incondizionata usiamo il beq sul registro x0, ottenendo `beq x0, x0, L1`. Oppure RARS mette a disposizione la pseudo-istruzione `j LABEL`.

Costrutto if-then-else

Partendo dal sorgente C:

```
if (i==j)
    f=g+h;
else
    f=g-h;
```

Dato il seguente mapping in memoria:

- $f \rightarrow x19$
- $g \rightarrow x20$
- $h \rightarrow x21$
- $i \rightarrow x22$
- $j \rightarrow x23$

Otteniamo in assembly dati:

```
bne x22,x23,ELSE    # Il bne in questi casi è più comodo
add x19,x20,x21      # Somma
beq x0,x0,ENDIF      # Salto dopo aver eseguito il codice if
ELSE: sub x19,x20,x21  # Sottrazione
ENDIF:
```

Quindi:

- Neghiamo la condizione logica
- Saltiamo all'ELSE se fosse vera

-
- Altrimenti mettiamo il codice della condizione vera dopo il bne
 - Infine saltiamo all'endif per evitare di eseguire il codice nell'else

Costrutto for

Dato

```
for (i=0; i<100; i++) {  
    // Corpo del ciclo  
}
```

Dato i -> x19

add x19,x0,x0 # Imposto x19 a x0

addi x20,x0,100 # Imposto x20 a 100, non abbiamo i salti immediati

FOR:

bge x19,x20,ENDFOR # Se i >= 100 finisco il for, altrimenti eseguo il
↪ corpo del ciclo

Corpo del ciclo

addi x19,x19,1 # Eseguo i++

j FOR # Salto incondizionato

ENDFOR:

Il ciclo while è analogo (senza incrementare i e cambiando la condizione).

Invece, il do-while vorrebbe il controllo della condizione solo alla fine.

SLT

slt REGISTRO_DEST REGISTRO_1 REGISTRO_2 e slti REGISTRO_DEST REGISTRO_1
IMMEDIATO modificano il registro destinazione scrivendo +1 in rd se rs1 < rs2, altrimenti scrive 0

Questa istruzione esegue il seguente codice C:

```
if (i<j)  
    k=1;  
else  
    k=0;
```

Questa istruzione può essere inserita prima dei salti per creare condizioni più sofisticate.

Esistono anche le versioni unsigned sltu e sltiu

Pseudo istruzioni

Sono istruzioni non disponibili nell'ISA di RISC-V ma sono messe a disposizione dall'assemblatore

Copia di registro in un altro

`mv x5, x6`

Sarebbe convertito come `addi x5, x6, 0`